12. visualization

Julia Gallucci HB.Sc, M.Sc, Ph.D.

Campbell Family Mental Health Research Institute, CAMH

Institute of Medical Science | University of Toronto

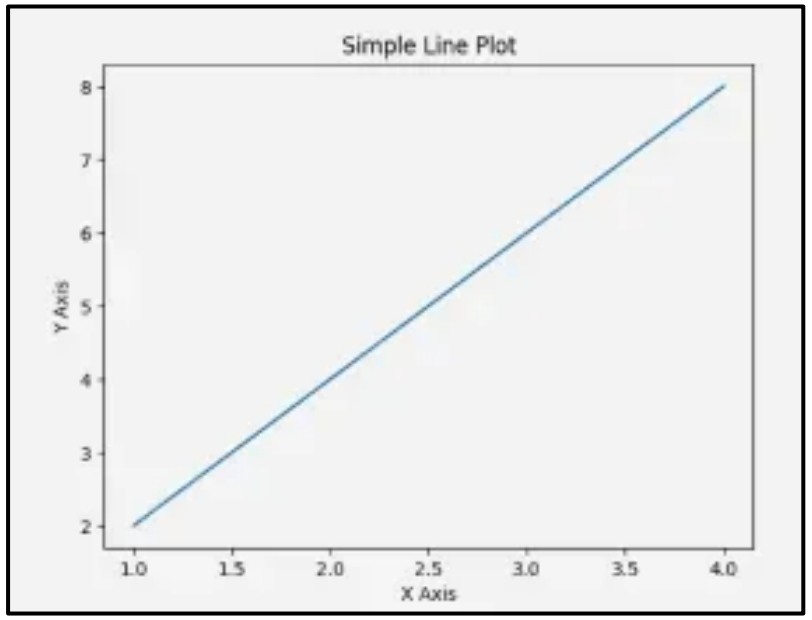
Learning objectives

- Understand how to plot in Python using visualization libraries like matplotlib and seaborn

Common types of plots

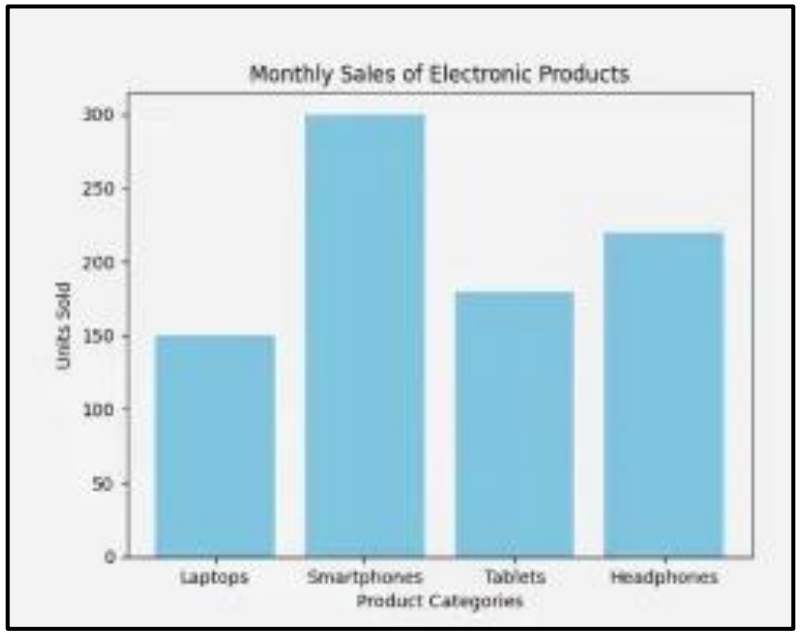
Common types of plots: **Line plot**

- A line plot connects data point with straight lines to show trends
- Common uses:
  - Tracking time series data (e.g., stock prices, temp)
  - Identifying patterns and trends in continuous data



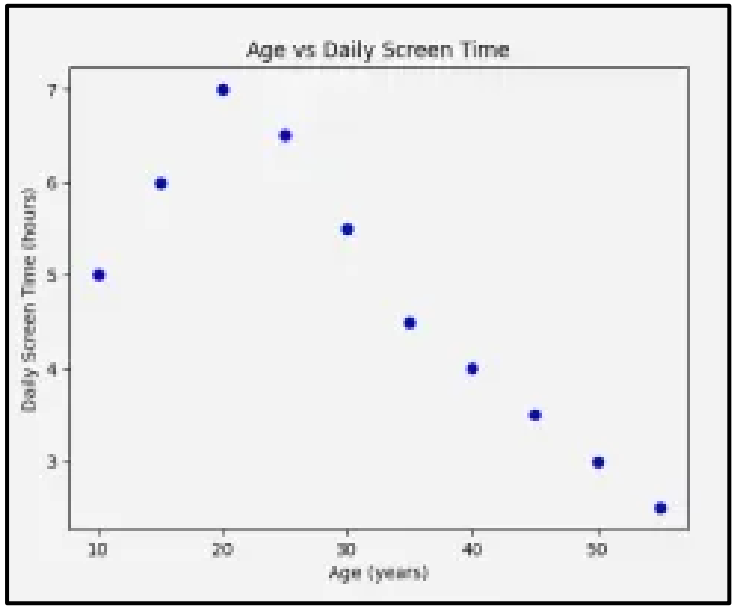
Common types of plots: **Bar plot**

- A bar plot uses rectangle bars to represent categorical data, with height of the bar indicating the value
- Common uses:
  - Comparing categories or groups
  - Showing distribution and frequency



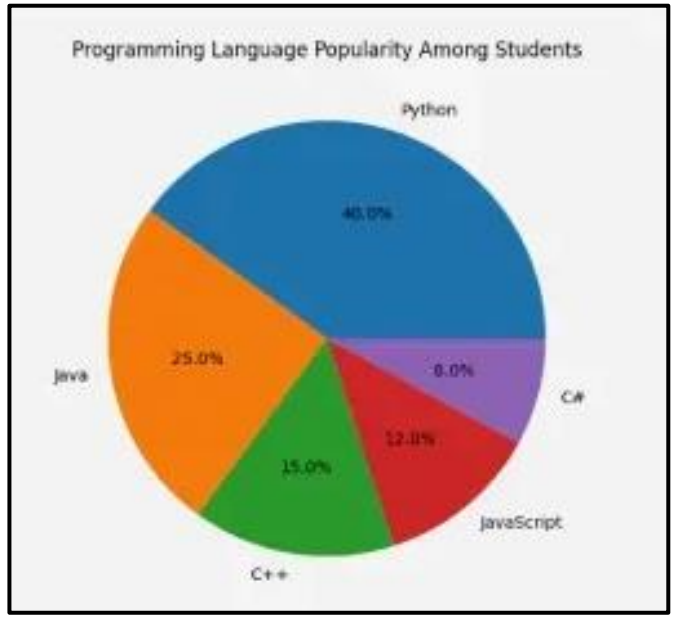
Common types of plots: **Scatter plot**

- A scatter plot visualizes relationships between two variables using individual data points
- Common uses:
  - Show relationship between two numerical variables
  - Identify trends, clusters or outliers
  - Visualizing correlations in data



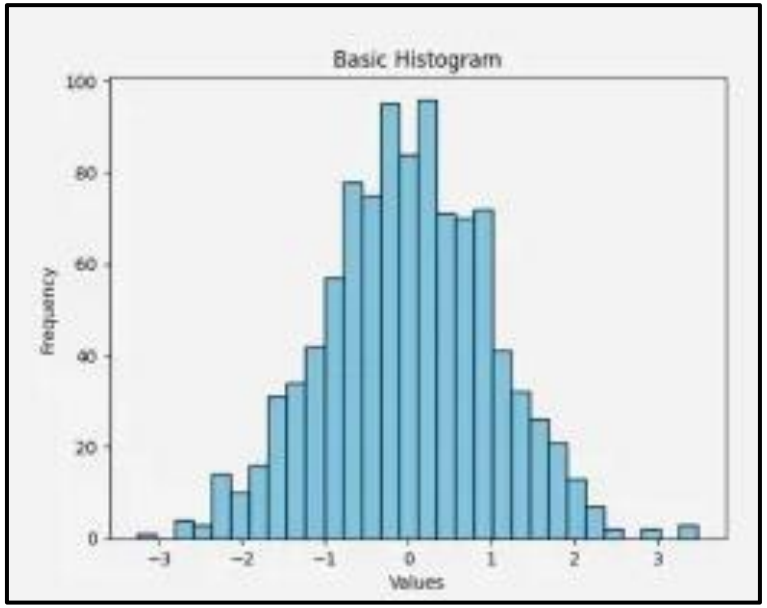
Common types of plots: **Pie plot**

- A pie plot represents data as a slice of a circle, showing proportions of a whole
- Common uses:
  - Showing proportions or % of distribution
  - Visualizing categorical data as parts of a whole
  - Comparing relative sizes of different categories



Common types of plots: **Histogram**

- A histogram represents the distribution of numerical data using bars, where the height of the bar indicates frequency
- Common uses:
  - Visualizing distribution of continuous data
  - Identifying patterns, skewness and outliers in data
  - Understanding frequency distribution in data



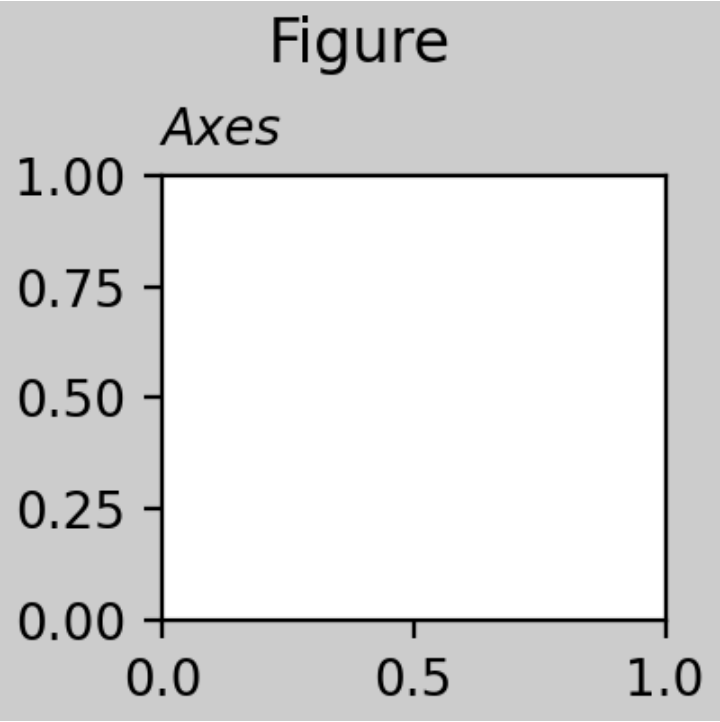
Matplotlib

- `Matplotlib` is the foundational data visualization library in Python
- `Pyplot` is a module within Matplotlib that provides an interface for making plots.
- When we import matplotlib, we really import `matplotlib.pyplot` using the alias `plt`

```
import matplotlib.pyplot as plt
```

The anatomy of a plot

- `Matplotlib` creates visuals that consist of one or more **Axes** in a **Figure**
- `Figure` is the overarching container that holds all plot elements, acting as the canvas for visualizations.
- `Axes` is the areas within the figure where data is plotted; each figure can contain multiple axes.



Plotting with matplotlib

- `Matplotlib` provides two ways to create visualizations

1. By having `pyplot` automatically create and manage Figures and Axes, keeping track of which Figure and Axes we are currently working on

2. Recommended: By taking an **object-oriented approach**, where we explicitly create Figures and Axes and modify them

Pyplot-style plotting

- `pyplot` style plotting is typically for quick, exploratory plots we don't plan on doing a lot of customizations too.
- For example,

```
import pandas as pd
airlines =
pd.read_csv('01_material/data/airlines.csv')
plt.plot(airlines['Delays_Weather'],
        airlines['Flights_Delayed'])
plt.show() #to display the figure
```

This produces a line plot by default!

Pyplot-style plotting

- When using pyplot, if you create more than one plot in the same cell, they will all be drawn on top of each other unless you clear or create a new figure.

```
plt.scatter(airlines['Delays_Weather'],
            airlines['Flights_Delayed'],
            marker = 's', # squares
            facecolor = '#fb1', # orange
            edgecolors='k') # black
plt.scatter(airlines['Delays_Security'],
            airlines['Flights_Delayed'],
            marker = 'o', # circles
            facecolor = '#abc', # blue
            edgecolors='k') # black
plt.show()
```

Object-oriented approach to plotting

- The object-oriented approach is the preferred method of plotting. In this approach, we use `subplots()` function to create plot objects and call methods to modify them

```
fig, ax = plt.subplots()
```

```
ax.scatter(airlines['Delays_Weather'],
              airlines['Flights_Delayed'],
              marker = 's', # squares
              facecolor = '#fb1', # orange
              edgecolors='k' , label = 'Weather Delays')
ax.scatter(airlines['Delays_Security'],
              airlines['Flights_Delayed'],
              marker = 'o', # circles
              facecolor = '#abc', # blue
              edgecolors='k', label = 'Security Delays’)
plt.show()
```

Adding labels, a title and legend

- This graph doesn't give much context! Let's add a title and axis labels.

```
ax.set_title('Specific Type Delays vs Overall Flight Delays')
ax.set_xlabel('Specific Delays')
ax.set_ylabel('Overall Delays')
plt.show()
```

- We can also add a legend using the `legend` method. Since we passed a label argument to our `scatter()` plots, legend uses those labels!

```
ax.legend()
plt.show()
```

Changing styles

- Matplotlib comes with a bunch of predefined styles! We can view available ones with `plt.style.available`
- Passing one of the options to `style.use()` makes it the aesthetic style for all new plots → Note it wont change the style of figures made prior

```
plt.style.available[5:10] #view some of the options
#Output
['classic', 'dark_background', 'fast',
'fivethirtyeight', 'ggplot']

# set style for new plots
plt.style.use('fivethirtyeight')

plt.show()
```

Note the style of our previously made figure will not change unless we rerun the code to generate it!

Other plot types

- `Matplotlib` offers more than just line plots and scatter plots → Many other ones like bar plots, histograms, pie plots…
- To create each the object-oriented way, we call the appropriate Axes method, for example `Axes.boxplot()` or `Axes.barh()` for a horizontal bar plot

```
small_airlines = airlines.head(n =10) # Let's work with
a small version of our data for simplicity
```

```
bar_fig, bar_ax = plt.subplots()
bar_ax.barh(small_airlines['Airport_Code'],
small_airlines['Late_Aircraft'])
bar_ax.set_title('Numbers of late Aircrafts for first 10
rows')
plt.show()
```

Subplots

- We can create multiple Axes in one Figure by passing `nrows` and `ncols` arguments to `subplots()`

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2)
```

```
ax1.barh(small_airlines['Airport_Code'],
         small_airlines['Late_Aircraft'])
ax2.scatter(small_airlines['Delays_Security'],
            small_airlines['Flights_Delayed'])
ax1.set_title('Numbers of late Aircrafts for each Airport')
ax2.set_title('Security Delays vs Overall Flight Delays')
plt.show()
```

This creates one figure with two plots stacked vertically.
Key point: `plt.subplots()` lets us create multiple plots in one figure, and each ax controls a different plot!

Saving plots

- To save a plot, use the Figure `savefig()` method, which supports common formats like PNG, PDF etc.
- Using bbox_inches = “tight” argument will make matplotlib try to figure out the dimensions of the plot and crop the image appropriately.

```
fig.savefig('example.png', bbox_inches = 'tight')
```

Clearing plots

- As we saw, successive method calls on an Axes object layer on graphics. To clear everything from an Axes, we can use `clear()` method.

```
ax1.clear() #removed the first plot
fig
```

```
ax2.clear() #removed the second plot
fig
```

- We can also reset our style back to the default if we want

```
plt.style.use('default')
```

Easier plotting with seaborn

- `seaborn` builds upon and complements matplot lib, producing nicer looking plots with less code !
- `seaborn` is typically given the alias `sns`

```
import seaborn as sns
```

Plotting with seaborn

- `seaborn` gives us two ways of structuring arguments to plot function

1. Specify the x and y axis columns

```
sns.scatterplot(x = airlines['Delays_Security'],
                y = airlines['Flights_Delayed'])
```

2. Specifying the data we are visualizing, then the x and y axis columns

Plotting with seaborn

- `seaborn` gives us two ways of structuring arguments to plot function

  1. Specify the x and y axis columns

  2. Specifying the data we are visualizing, then the x and y axis columns

```
sns.scatterplot(data = airlines,
                x = 'Delays_Security',
                y = 'Flights_Delayed')
```

Plotting with seaborn

- Using Matplotlib's pyplot interface, we can specify axis labels, titles etc. and also set themes!

```
sns.set_theme() #set to seaborns default theme
sns.scatterplot(data = airlines,
                x = 'Delays_Security',
                y = 'Flights_Delayed')
plt.xlabel('Security delays')
plt.ylabel('Overall Flight delays')
```

Quiz: visualization

1. **What does Matplotlib provide in python?**

A. Data manipulation

B. Data visualization

C. Text processing

D. Machine learning

Quiz: visualization

2. What is the purpose of subplot() function in Matplotlb?

A. Create multiple plots in one figure

B. Set the subplot title

C. Add subplots to a figure

D. Adjust subplot spacing

Quiz: visualization

3.What is the correct way to add a title to a Matplotlib plot?

A. add_title(“My Plot”)

B. set_title(“My Plot”)

C. title(“My Plot”)

D. plot_title(“My Plot”)

Quiz: visualization

4. How would you plot a simple line graph using Matplotlib?

A. plot(x_values, y_values)

B. lineplot(x_values, y_values

C. draw_line(x_values, y_values

D. create_line_graph(x_values, y_values

Exercise

- Try This on Your Own
  - Load the airlines.csv dataset into a pandas DataFrame.
  - Create two different plots of your choice (e.g., line, scatter, bar, histogram).
  - Customize your plots – change colors, markers, line styles, etc.
  - Add titles and axis labels for each plot.
  - Save your plots as PNG files (e.g., plot1.png and plot2.png)

matplotlib Cheat sheet

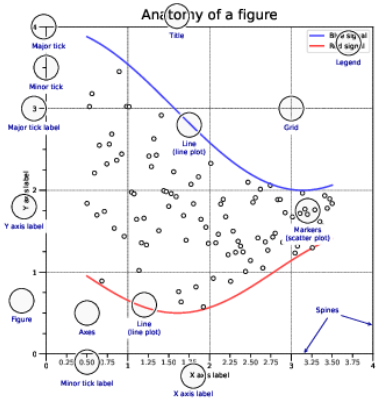
Version 3.9.4

Quick start

API

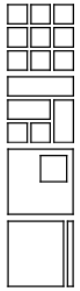
```
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt


X = np.linspace(0, 2*np.pi, 100)
Y = np.cos(X)

fig, ax = plt.subplots()
ax.plot(X, Y, color='green')

fig.savefig("figure.pdf")
plt.show()
```

Anatomy of a figure



Subplots layout

API



```
subplot[s](rows, cols, …)
fig, axs = plt.subplots(3, 3)

G = gridspec(rows,cols, …)
ax = G[0, :]

ax.inset_axes(extent)

d=make_axes_locatable(ax)
ax = d.new_horizontal('10%')
```

Getting help

- matplotlib.org
- github.com/matplotlib/matplotlib/issues
- discourse.matplotlib.org
- stackoverflow.com/questions/tagged/matplotlib
- https://gitter.im/matplotlib/matplotlib
- twitter.com/matplotlib
- Matplotlib users mailing list

Basic plots

`plot([X], Y, [fmt], …)` API
X, **Y**, fmt, color, marker, linestyle

`scatter(X, Y, …)` API
X, **Y**, [s]izes, [c]olors, marker, cmap

`bar[h](x, height, …)` API
x, **height**, width, bottom, align, color

`imshow(Z, …)` API
Z, cmap, interpolation, extent, origin

`contour[f]([X], [Y], Z, …)` API
X, Y, **Z**, levels, colors, extent, origin

`pcolormesh([X], [Y], Z, …)` API
X, Y, **Z**, vmin, vmax, cmap

`quiver([X], [Y], U, V, …)` API
X, Y, **U**, **V**, C, units, angles

`pie(X, …)` API
Z, explode, labels, colors, radius

`text(x, y, text, …)` API
x, **y**, **text**, va, ha, size, weight, transform

`fill[_between][x](…)` API
X, Y1, Y2, color, where

Advanced plots

`step(X, Y, [fmt], …)` API
X, **Y**, fmt, color, marker, where

`boxplot(X, …)` API
X, notch, sym, bootstrap, widths

`errorbar(X,Y,xerr,yerr, …)` API
X, **Y**, xerr, yerr, fmt

`hist(X, bins, …)` API
X, bins, range, density, weights

`violinplot(D, …)` API
D, positions, widths, vert

`barbs([X], [Y], U, V, …)` API
X, Y, **U**, **V**, C, length, pivot, sizes

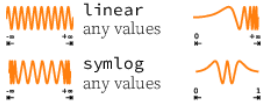
`eventplot(positions, …)` API
positions, orientation, lineoffsets

`hexbin(X, Y, C, …)` API
X, **Y**, C, gridsize, bins

Scales

API

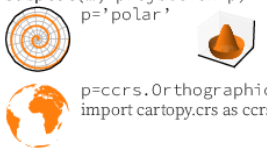
```
ax.set_[xy]scale(scale, …)
```



Projections

API

```
subplot(…, projection=p)
```

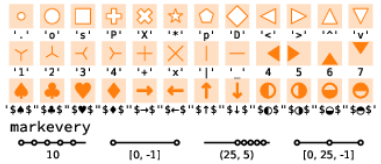


Lines

API

Markers

API



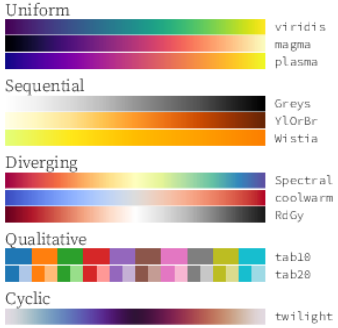
Colors

API

Colormaps

API

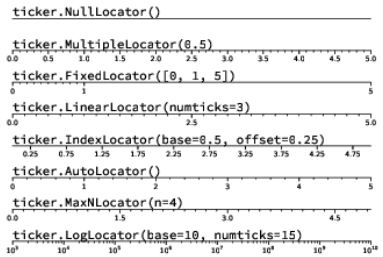
```
plt.get_cmap(name)
```



Tick locators

API

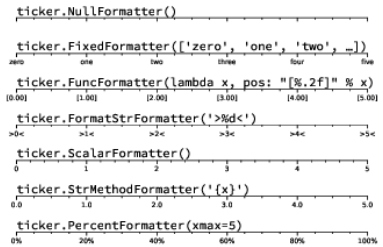
```
from matplotlib import ticker
ax.[xy]axis.set_[minor|major]_locator(locator)
```



Tick formatters

API

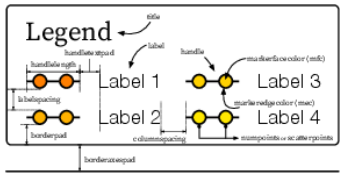
```
from matplotlib import ticker
ax.[xy]axis.set_[minor|major]_formatter(formatter)
```



Ornaments

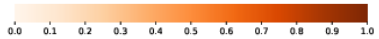
API

`ax.legend(…)`
handles, labels, loc, title, frameon



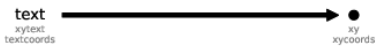
API

`ax.colorbar(…)`
mappable, ax, cax, orientation



API

`ax.annotate(…)`
text, **xy**, **xytext**, xycoords, textcoords, arrowprops



Event handling

API

```
fig, ax = plt.subplots()
def on_click(event):
  print(event)
fig.canvas.mpl_connect(
  'button_press_event', on_click)
```

Animation

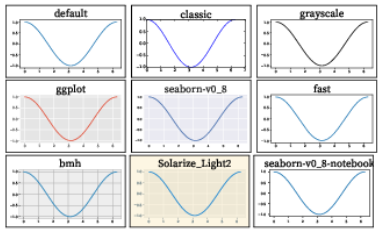
API

```
import matplotlib.animation as mpla

T = np.linspace(0, 2*np.pi, 100)
S = np.sin(T)
line, = plt.plot(T, S)
def animate(i):
    line.set_ydata(np.sin(T+i/50))
anim = mpla.FuncAnimation(
    plt.gcf(), animate, interval=5)
plt.show()
```

Styles

API

```
plt.style.use(style)
```



Quick reminder

```
ax.grid()
ax.set_[xy]lim(vmin, vmax)
ax.set_[xy]label(label)
ax.set_[xy]ticks(ticks, [labels])
ax.set_[xy]ticklabels(labels)
ax.set_title(title)
ax.tick_params(width=10, …)
ax.set_axis_[on|off]()

fig.suptitle(title)
fig.tight_layout()
plt.gcf(), plt.gca()
mpl.rc('axes', linewidth=1, …)
[fig|ax].patch.set_alpha(0)
text=r'$\frac{-e^{i\pi}}{2^n}$'
```

Keyboard shortcuts

API

ctrl+s	Save	ctrl+w	Close plot
r	Reset view	f	Fullscreen 0/1
f	View forward	b	View back
p	Pan view	o	Zoom to rect
x	X pan/zoom	y	Y pan/zoom
g	Minor grid 0/1	G	Major grid 0/1
l	X axis log/linear	L	Y axis log/linear

Ten simple rules

READ

1. Know your audience
2. Identify your message
3. Adapt the figure
4. Captions are not optional
5. Do not trust the defaults
6. Use color effectively
7. Do not mislead the reader
8. Avoid "chartjunk"
9. Message trumps beauty
10. Get the right tool